

Prompt:

“ Correct/ modify these prompts to make them more precise:

Prompt 1 — Bien cadrer le projet avant de commencer à coder

On va maintenant commencer le prototype sur le sujet « **Admission d'un patient** ».

L'objectif est de montrer concrètement l'interopérabilité entre un serveur FHIR et un système utilisant HL7 v2.

Le scénario général sera donc :

HAPI FHIR R4 → Patient / Encounter → transformation → message HL7 v2.5.1 ADT^A01

Le serveur FHIR utilisé pour les tests sera :

<https://hapi.fhir.org/baseR4>

Je veux rester sur quelque chose de simple et réalisable dans le temps disponible. Le but n'est pas de recréer tout un système d'information hospitalier, mais de développer un prototype qui permet de démontrer les points importants du cours.

L'application devra au minimum permettre :

- d'interagir réellement avec le serveur FHIR ;
- de récupérer les données Patient et Encounter depuis HAPI, sans données métier codées en dur ;
- d'avoir une vue compréhensible pour un utilisateur métier ;
- d'afficher les ressources FHIR utilisées ;
- de montrer le résultat de la transformation en HL7 v2 ;
- de montrer au moins un exemple clair de mapping entre un concept source et un concept cible ;
- de gérer les erreurs principales ;
- d'avoir un minimum de traçabilité des échanges avec le serveur ;
- de tester l'application avec plusieurs patients sans modifier le code.

Pour le MVP, je veux rester principalement sur les ressources **Patient** et **Encounter**.

Le scénario fonctionnel est le suivant :

chercher un Patient existant → le sélectionner → vérifier ses données → créer un Encounter correspondant à son admission → récupérer la réponse réelle du serveur → afficher les

données sous une forme métier → générer le message ADT^A01 → afficher les mappings utilisés.

Pour représenter l'admission, l'Encounter devra notamment avoir :

- `status = in-progress`
- `class.code = IMP`
- `subject.reference = Patient/{id}`
- `period.start` correspondant à la date et à l'heure de l'admission.

Le message HL7 v2.5.1 ADT^A01 devra contenir au minimum :

MSH

EVN

PID

PV1

Je veux aussi éviter d'ajouter des éléments qui ne sont pas nécessaires au prototype. Donc :

- pas de Patient fictif directement écrit dans le code ;
- pas d'identifiant Patient fixe ;
- pas de fausse réponse HAPI ;
- pas de données FHIR inventées ;
- pas de base de données supplémentaire ;
- pas de microservices ;
- pas de Location, Practitioner ou Organization sauf si on découvre qu'ils sont vraiment nécessaires.

S'il existe déjà un projet, garde le framework actuel. Sinon, propose-moi d'abord l'architecture la plus simple adaptée à ce prototype.

Avant de coder, commence par :

1. analyser l'état actuel du projet ;
2. proposer l'architecture minimale ;
3. me dire quels fichiers devront être créés ou modifiés ;
4. identifier les points qu'on doit encore vérifier ;
5. proposer un ordre logique d'implémentation ;
6. identifier les risques techniques possibles, notamment CORS et la disponibilité du serveur HAPI.

Pour l'instant, ne code rien. Je veux d'abord valider cette organisation.

Pour tout élément dont on n'est pas encore sûr, indique clairement « **À VÉRIFIER** ».

Prompt 2 — Connexion réelle au serveur FHIR

Maintenant que l'organisation du projet est claire, commence uniquement par mettre en place la communication avec le serveur FHIR.

Ne développe pas encore toute l'interface.

La Base URL est :

<https://hapi.fhir.org/baseR4>

Je préfère que cette URL soit définie une seule fois dans la configuration du projet et qu'elle ne soit pas répétée un peu partout dans le code.

Pour cette première étape, je veux pouvoir :

- vérifier que le serveur répond avec `GET /metadata` ;
- rechercher des Patients avec une requête réellement acceptée par HAPI ;
- récupérer un Patient précis avec `GET /Patient/{id}` ;
- interpréter correctement le résultat d'une recherche FHIR à partir de `Bundle.entry[ ].resource`.

Quelques points importants :

Les Patients doivent réellement venir de HAPI. Je ne veux pas de tableau local avec de faux patients ou de fallback vers des données simulées si le serveur ne répond pas.

Avant d'utiliser une ressource, vérifie son `resourceType`.

Il faut aussi tenir compte du fait que certains champs FHIR sont optionnels. Le code ne doit donc pas supposer automatiquement qu'un nom, un identifiant ou une date de naissance sera toujours présent.

Je veux également distinguer correctement :

- une erreur HTTP ;
- une réponse FHIR invalide ;
- un `OperationOutcome` ;
- une recherche qui ne retourne aucun résultat.

Une fois cette partie terminée, donne-moi un petit bilan avec :

- les endpoints réellement utilisés ;
- les paramètres de recherche choisis ;
- le type de réponse observé ;
- les champs Patient que le code utilise réellement ;
- les hypothèses qui ont été confirmées ou qui étaient finalement fausses.

Fais aussi quelques tests simples :

- serveur accessible ;
- recherche qui retourne un Bundle ;
- lecture d'un Patient ;
- recherche sans résultat ;
- Patient inexistant ;
- erreur réseau ou endpoint incorrect.

Pour chaque cas, note simplement :

test → résultat attendu → résultat obtenu → correction si nécessaire

Si HAPI se comporte différemment de ce qu'on avait prévu, ne cherche pas à cacher le problème. Explique ce qui se passe réellement et adapte le prototype de la manière la plus simple possible.

Prompt 3 — Créer réellement l'admission avec Encounter

On peut maintenant ajouter le parcours d'admission.

Le Patient utilisé doit obligatoirement être un Patient que nous avons récupéré depuis HAPI.

Quand l'utilisateur confirme l'admission, crée réellement une ressource **Encounter** avec les informations suivantes :

```
resourceType = Encounter
status = in-progress
class.system = http://terminology.hl7.org/CodeSystem/v3-ActCode
class.code = IMP
subject.reference = Patient/{idFHIR}
period.start = date/heure de l'admission
```

La création doit se faire avec :

POST /Encounter

Quelques règles importantes :

- utilise l'id du Patient réellement sélectionné ;
- ne mets jamais un id du type **Patient/123** directement dans le code ;
- **period.start** doit être généré au moment où l'utilisateur effectue l'admission ;
- après le POST, utilise l'Encounter réellement retourné par HAPI ;
- affiche son id ;
- vérifie que **Encounter.subject.reference** correspond bien au Patient choisi ;

- si le serveur refuse la création, l'application ne doit surtout pas afficher un faux succès.

Pour le moment, reste sur ce scénario simple. N'ajoute pas encore :

- création de Patient ;
- Location ;
- Practitioner ;
- Organization ;
- modification de l'Encounter ;
- transfert ou sortie du patient.

Teste au minimum les cas suivants :

- sélection d'un Patient réel puis création de l'Encounter ;
- vérification de `subject.reference` ;
- `status = in-progress` ;
- `class.code = IMP` ;
- présence de `period.start` ;
- comportement de l'application si le POST est refusé.

À la fin, fais-moi un petit compte rendu sous cette logique :

hypothèse → requête envoyée → réponse obtenue → validation → correction éventuelle

Si la structure réellement retournée par HAPI n'est pas exactement celle qu'on avait imaginée, analyse d'abord la réponse avant de modifier le code.

Prompt 4 — Créer la vue métier et la vue FHIR

On peut maintenant travailler sur l'affichage.

Pour cette étape, je veux uniquement deux vues.

1. Vue métier

Cette vue doit pouvoir être comprise par un agent hospitalier qui ne connaît pas forcément FHIR ou HL7.

Elle peut afficher, lorsqu'elles sont disponibles :

- l'identité du patient ;
- sa date de naissance ;
- son sexe administratif ;
- son identifiant métier ;

- le statut de l'admission ;
- la date et l'heure de l'admission ;
- l'identifiant de l'Encounter.

Si une donnée n'existe pas dans la ressource FHIR, ne l'invente surtout pas. Affiche par exemple « **Non disponible** », ou ne montre pas le champ si c'est plus lisible.

2. Vue FHIR

Dans cette vue, je veux pouvoir consulter exactement les ressources réellement utilisées :

- Patient ;
- Encounter.

Affiche le JSON de façon lisible et correctement formatée.

Je veux notamment pouvoir montrer visuellement la relation entre les deux ressources :

Encounter.subject.reference

↓

Patient/{id}

Les deux vues doivent être basées sur les mêmes objets provenant du serveur.

Il ne faut pas reconstruire une deuxième version du Patient avec des données fictives uniquement pour l'affichage métier.

Pour vérifier que l'application reste générique, teste-la avec deux Patients différents récupérés sur HAPI.

Regarde notamment si certains champs sont présents chez un Patient et absents chez l'autre, et vérifie que l'interface s'adapte correctement sans changer le code.

L'idée est aussi de montrer qu'une même information peut avoir une représentation technique FHIR et une représentation métier plus facile à comprendre.

Prompt 5 — Transformer Patient + Encounter en HL7 v2 ADT^A01

On passe maintenant à la partie transformation.

À partir du **Patient et de l'Encounter réellement obtenus depuis HAPI**, génère un message :

HL7 v2.5.1 — ADT^A01

Le message doit au minimum contenir :

MSH
EVN
PID
PV1

Je veux que la logique de transformation soit séparée de l'interface pour éviter de mélanger l'affichage et les règles de mapping.

Pour commencer, vérifie puis implémente les mappings suivants lorsqu'ils sont applicables :

Patient.identifier → PID-3
Patient.name → PID-5
Patient.birthDate → PID-7
Patient.gender → PID-8
Encounter.class → PV1-2
Encounter.identifier → PV1-19 si disponible
Encounter.period.start → PV1-44

Pour chaque mapping, je veux pouvoir comprendre :

- quel est le champ FHIR source ;
- quel est le champ HL7 cible ;
- quelle règle est appliquée ;
- quel type de transformation est utilisé ;
- ce que l'on fait si la donnée source est absente.

On peut distinguer quatre types :

- correspondance directe ;
- transformation syntaxique ;
- mapping sémantique ;
- correspondance non déterminée.

Par exemple, pour la date de naissance :

Patient.birthDate
2002-04-10
↓
transformation syntaxique
↓
PID-7
20020410

Ne crée aucune donnée métier uniquement pour avoir un message HL7 plus rempli.

Si une information indispensable manque réellement, je préfère avoir un message d'erreur clair expliquant quel champ manque plutôt qu'une valeur inventée.

Ajoute ensuite une vue dans laquelle on peut lire le message HL7 généré, de préférence avec une police monospace.

Cette vue doit toujours être basée sur le Patient et l'Encounter actuellement sélectionnés.

Pendant les tests, vérifie notamment :

- `MSH-9 = ADT^A01` ;
- présence de MSH, EVN, PID et PV1 ;
- conversion d'une date `YYYY-MM-DD` vers `YYYYMMDD` ;
- conversion correcte d'un datetime FHIR vers le format HL7 ;
- identité provenant du bon Patient ;
- informations d'admission provenant du bon Encounter.

En cas d'erreur de transformation, indique :

valeur source → **valeur attendue** → **valeur produite** → **cause** → **correction** → **nouveau test**

Prompt 6 — Montrer clairement le mapping sémantique

Je veux maintenant ajouter une vue qui montre clairement qu'on ne fait pas uniquement une conversion de format, mais aussi des mappings entre des terminologies différentes.

Premier exemple : sexe administratif

La source est :

`Patient.gender`

Les valeurs possibles que nous pouvons rencontrer sont notamment :

male
female
unknown
other

La cible est :

`PID-8`

Pour les correspondances que nous avons confirmées :

male → M
female → F
unknown → U

Pour **other**, ne choisis pas automatiquement une lettre uniquement parce qu'elle semble logique.

Si on ne peut pas justifier une correspondance exacte dans le mapping retenu, indique plutôt :

Correspondance non déterminée / ambiguë

Je préfère rendre cette limite visible plutôt que faire une conversion silencieuse.

Deuxième exemple : classe de séjour

Nous avons également :

FHIR Encounter.class.code = IMP

↓
HL7 v2 PV1-2 = I

Je veux une petite vue ou un tableau qui permet de visualiser ce type de transformation avec par exemple :

Source | Système source | Règle | Cible | Type de mapping

Exemples :

male | FHIR AdministrativeGender | male → M | M | équivalence
IMP | v3-ActCode | IMP → I | I | mapping sémantique

On peut aussi ajouter la conversion d'une date comme exemple de **transformation syntaxique**, pour montrer la différence entre changer la forme d'une donnée et changer de système terminologique.

Pour chaque mapping présenté, vérifie :

- le système source ;
- le concept représenté ;
- le système cible ;
- si le sens reste le même ;
- s'il existe une perte d'information ou une ambiguïté.

Si une ambiguïté existe, elle doit être visible dans l'interface.

Je veux éviter les mappings basés uniquement sur le fait que deux valeurs « se ressemblent ».

Prompt 7 — Vérification finale, erreurs et traçabilité

Le parcours principal fonctionne maintenant. Avant de considérer le prototype comme terminé, fais une dernière revue complète par rapport au cahier des charges.

Gestion des erreurs

Vérifie au minimum les situations suivantes :

- serveur FHIR indisponible ;
- erreur HTTP 4xx ;
- erreur HTTP 5xx ;
- Patient inexistant ;
- Bundle sans `entry` ;
- `resourceType` inattendu ;
- réponse `OperationOutcome` ;
- création Encounter refusée ;
- champ nécessaire à une transformation absent ;
- mapping sémantique impossible ou ambigu.

Dans aucun de ces cas l'application ne doit afficher un faux message de succès.

Traçabilité

Ajoute une petite traçabilité technique des échanges avec le serveur avec, au minimum :

- date et heure ;
- méthode HTTP ;
- endpoint ;
- type de ressource ;
- code HTTP ;
- résultat ;
- erreur éventuelle.

Évite par contre d'enregistrer inutilement les informations personnelles du Patient dans les logs.

Les logs peuvent simplement être affichés dans une zone technique discrète. Le but n'est pas de construire un système complet de supervision.

Test avec plusieurs Patients

Refais ensuite tout le parcours avec au moins deux Patients différents provenant de HAPI.

Je veux vérifier que :

- aucune donnée Patient n'est codée en dur ;
- l'application ne dépend pas d'un id précis ;
- les champs optionnels sont correctement gérés ;
- le message HL7 change réellement en fonction du Patient et de l'Encounter ;
- les mappings restent cohérents.

Matrice finale

À la fin, prépare une petite matrice :

Exigence | Implémentée ? | Test réalisé | Résultat | Correction restante

Elle doit couvrir au minimum :

- connexion réelle à FHIR ;
- utilisation d'un Patient réel ;
- création réelle de l'Encounter ;
- vue métier ;
- vue FHIR ;
- vue HL7 ;
- vue des mappings ;
- gestion des erreurs ;
- traçabilité ;
- test avec plusieurs Patients.

Relecture critique

Avant de conclure, cherche aussi activement dans le projet :

- une donnée métier codée en dur ;
- une valeur inventée ;
- un mapping qui n'est pas justifié ;
- une fonctionnalité inutile ;
- du code dupliqué ;
- une erreur qui serait masquée ;
- une mauvaise relation Patient / Encounter ;
- une confusion entre donnée métier et donnée technique.

Pour chaque problème trouvé, indique :

problème → cause → correction → test de non-régression

Enfin, considère le prototype comme **GO** uniquement si les principales exigences peuvent réellement être démontrées avec les données obtenues depuis HAPI.”

Résultat:

Prompt 1 — Cadrer la réalisation avant de coder

Nous allons maintenant réaliser le prototype défini dans notre cahier des charges pour le sujet « **Admission d'un patient** ».

Contexte

Le prototype doit démontrer une interopérabilité réelle entre :

Serveur HAPI FHIR R4 → ressources Patient/Encounter →
transformation → HL7 v2.5.1 ADT^A01

Serveur FHIR :

<https://hapi.fhir.org/baseR4>

Le développement doit rester réalisable dans le cadre d'un prototype court.
Il ne s'agit pas de construire un SI hospitalier complet.

Exigences obligatoires de l'évaluation

L'application doit comporter :

1. une connexion et des interactions réelles avec le serveur FHIR ;
2. aucune donnée métier Patient/Encounter codée en dur ;
3. une vue métier compréhensible par un utilisateur ;
4. une vue affichant les ressources FHIR réellement utilisées ;
5. une vue affichant la conversion HL7 v2 ;
6. une vue montrant explicitement au moins un mapping terminologique
source → cible ;
7. une gestion minimale des erreurs ;
8. une traçabilité minimale des interactions ;
9. la possibilité de tester le prototype avec plusieurs patients sans modifier
le code.

Périmètre MVP

Ressources principales :

- Patient
- Encounter

Workflow attendu :

chercher/sélectionner un Patient existant → vérifier la
ressource → créer un Encounter d'admission → utiliser la

réponse réelle du serveur → produire une vue métier →
produire ADT^A01 → afficher les mappings

L'admission sera représentée par un **Encounter** avec notamment :

- `status = in-progress`
- `class.code = IMP`
- `subject.reference = Patient/{id}`
- `period.start = date/heure réelle de l'admission`

Le message cible est **HL7 v2.5.1 ADT^A01**, avec au minimum :

- **MSH**
- **EVN**
- **PID**
- **PV1**

Contraintes

- Ne crée pas de Patient fictif codé dans le code.
- Ne code pas un identifiant Patient particulier.
- Ne simule pas une réponse provenant de HAPI.
- N'invente aucune donnée FHIR absente.
- Pas de base de données supplémentaire.
- Pas de microservices.
- Pas de **Location**, **Practitioner** ou **Organization** sauf nécessité démontrée.
- Ne change pas de framework si un projet existe déjà.
- Si aucun projet n'existe, propose d'abord l'architecture technique la plus simple avant de coder.

Avant toute implémentation

1. analyse l'état actuel du projet ;
2. propose l'architecture minimale ;
3. liste les fichiers à créer ou modifier ;
4. identifie les hypothèses encore à vérifier ;
5. propose l'ordre d'implémentation ;
6. identifie les risques techniques, notamment CORS et disponibilité du serveur.

Ne développe rien tant que ce plan n'est pas présenté.

Toute hypothèse non confirmée doit être marquée **À VÉRIFIER**.

Pourquoi il est important : il montre que tu ne demandes pas directement du code. Tu définis **contexte, périmètre, architecture, contraintes et hypothèses**.

Prompt 2 — Connexion FHIR réelle et vérification avec HAPI

Une fois le plan accepté :

Implémente maintenant uniquement la **couche d'accès au serveur FHIR**, sans encore construire toute l'interface.

Base URL :

<https://hapi.fhir.org/baseR4>

La Base URL doit être centralisée dans une configuration et non répétée dans le code.

Fonctions minimales attendues

1. vérifier le serveur avec :

`GET /metadata`

2. rechercher des Patients avec une requête réellement supportée par HAPI ;
3. lire un Patient précis avec :

`GET /Patient/{id}`

4. interpréter correctement les résultats de recherche FHIR :

`Bundle.entry[ ].resource`

Contraintes

- Les Patients doivent venir réellement du serveur HAPI.
- Aucun tableau local de Patients fictifs.
- Aucun fallback silencieux vers des données simulées.
- Vérifier `resourceType` avant utilisation.
- Ne pas considérer automatiquement qu'un champ optionnel existe.
- Reconnaître une réponse FHIR `OperationOutcome`.
- Distinguer erreur HTTP, réponse FHIR invalide et absence de résultat.

Vérifications demandées

Après implémentation, indique précisément :

- les endpoints réellement utilisés ;
- les paramètres de recherche utilisés ;
- un exemple de type de réponse observé ;
- les champs Patient effectivement exploités ;
- les hypothèses qui ont été confirmées ou infirmées.

Tests

Tester au minimum :

- serveur accessible ;
- recherche retournant un **Bundle** ;
- lecture d'un Patient ;
- recherche sans résultat ;
- identifiant Patient inexistant ;
- erreur réseau ou endpoint incorrect.

Pour chaque test :

Test → résultat attendu → résultat obtenu → correction éventuelle

Si un comportement réel du serveur diffère du cahier des charges, ne contourne pas silencieusement le problème : explique l'écart puis propose une correction minimale.

Ici, ton journal de prompts montre clairement :

hypothèse → test sur HAPI → observation → correction.

Prompt 3 — Réaliser la vraie admission avec Encounter

Ensuite :

Ajoute maintenant le workflow d'**admission réelle d'un Patient existant**.

Le Patient doit avoir été obtenu depuis HAPI.

Lorsque l'utilisateur confirme l'admission, créer réellement une ressource **Encounter** avec :

resourceType = Encounter

status = in-progress

class.system = <http://terminology.hl7.org/CodeSystem/v3-ActCode>

```
class.code = IMP
subject.reference = Patient/{idFHIR}
period.start = date/heure réelle de l'action d'admission
```

Envoyer la ressource avec :

POST /Encounter

Exigences

- Utiliser l'id du Patient réellement sélectionné.
- Ne jamais coder **Patient/123** ou un id équivalent en dur.
- La date d'admission doit être générée au moment de l'action.
- Après le **POST**, utiliser l'Encounter réellement retourné par HAPI.
- Afficher son **id**.
- Vérifier **Encounter.subject.reference**.
- Ne pas annoncer un succès si le serveur refuse la création.

Ne pas ajouter au MVP

- création Patient ;
- Location séparée ;
- Practitioner ;
- Organization ;
- mise à jour Encounter ;
- sortie ou transfert.

Tests à exécuter

1. Patient réel sélectionné → Encounter créé ;
2. **Encounter.subject.reference** correspond au Patient ;
3. **status = in-progress** ;
4. **class.code = IMP** ;
5. **period.start** présent ;
6. serveur refusant le POST → message d'erreur et aucun faux succès.

Après implémentation, présente :

Hypothèse → requête réelle → réponse réelle → validation → correction éventuelle.

Si HAPI retourne une structure différente de celle prévue, analyse-la avant de modifier le code.

Prompt 4 — Construire les deux premières vues : métier + FHIR

Construis maintenant uniquement les deux premières vues demandées par l'évaluation.

Vue 1 — Vue métier

Elle doit être compréhensible par un agent hospitalier ne connaissant ni FHIR ni HL7.

Afficher uniquement les informations réellement disponibles, par exemple :

- identité ;
- date de naissance si présente ;
- sexe administratif si présent ;
- identifiant métier si disponible ;
- statut de l'admission ;
- date/heure d'admission ;
- identifiant de l'Encounter.

Un champ absent doit être présenté comme indisponible ou omis, jamais inventé.

Vue 2 — Ressources FHIR

Afficher exactement les ressources réellement utilisées :

- Patient ;
- Encounter.

Le JSON doit être formaté et lisible.

Je veux pouvoir démontrer visuellement :

Encounter.subject.reference

↓

Patient/{id}

Contraintes

- Les deux vues doivent utiliser les mêmes objets reçus du serveur.
- Ne recrée pas des copies métier avec des valeurs fictives.
- Ne masque pas la source de la donnée.
- Conserve une séparation claire entre représentation métier et JSON FHIR.

Vérification

Choisis deux Patients différents et confirme que la vue se met à jour en fonction des données réellement reçues, sans modification du code.

Signale les différences observées entre les deux Patients, notamment les champs optionnels absents.

Ce prompt démontre aussi une idée du cours :

même information, représentation différente selon le besoin utilisateur.

Prompt 5 — Transformation FHIR → HL7 v2 ADT^A01

Celui-ci est très important.

Implémente maintenant la transformation des ressources FHIR réellement obtenues vers le format cible :

HL7 v2.5.1 — ADT^A01

Le message doit contenir au minimum :

MSH

EVN

PID

PV1

Le moteur de transformation doit être séparé de l'interface.

Mappings principaux

Examiner et implémenter uniquement les mappings confirmés :

Patient.identifier → PID-3

Patient.name → PID-5

Patient.birthDate → PID-7

Patient.gender → PID-8

Encounter.class → PV1-2

Encounter.identifier → PV1-19 si disponible

Encounter.period.start → PV1-44

Pour chaque mapping, indiquer dans le code ou dans une structure de mapping :

- champ FHIR source ;
- champ HL7 cible ;
- règle de transformation ;
- type de transformation ;
- comportement si la source est absente.

Utiliser quatre catégories :

- **correspondance directe/exacte ;**
- **transformation syntaxique ;**
- **mapping sémantique ;**
- **correspondance non déterminée.**

Exemple :

Patient.birthDate

2002-04-10

↓

transformation syntaxique

↓

PID-7

20020410

Ne jamais créer une valeur métier uniquement pour rendre le message HL7 complet.

Si une information réellement requise manque, afficher une erreur de transformation expliquant précisément le champ source absent.

Vue HL7

Ajouter une vue affichant le message généré dans une zone monospace.

La vue doit être produite à partir du Patient et de l'Encounter réellement utilisés.

Tests

Vérifier notamment :

- **MSH-9 = ADT^A01 ;**
- présence de MSH / EVN / PID / PV1 ;
- date **YYYY-MM-DD → YYYYMMDD ;**
- date/heure FHIR → timestamp HL7 ;
- identité issue du bon Patient ;
- données séjour issues du bon Encounter.

Pour chaque échec :

source observée → valeur attendue → valeur produite → cause
→ correction → retest.

Prompt 6 — Mapping sémantique et vue terminologie

Ajoute maintenant la vue obligatoire démontrant explicitement un **alignement** ou **changement de terminologie**.

Cette vue ne doit pas simplement afficher la chaîne finale : elle doit permettre de comprendre la transformation.

Mapping 1 — sexe administratif

Source :

Patient.gender

Examiner :

male
female
unknown
other

Cible :

PID-8

Retenir les correspondances confirmées :

male → M
female → F
unknown → U

Pour **other**, ne force pas automatiquement une correspondance si la relation exacte n'est pas univoque dans le mapping normatif retenu.

Afficher dans ce cas :

Correspondance non déterminée / ambiguë

plutôt que traduire silencieusement.

Mapping 2 — classe de séjour

Examiner également :

FHIR Encounter.class.code = IMP

↓

HL7 v2 PV1-2 = I

La vue doit présenter un tableau du type :

Source | Système source | Règle | Cible | Type de mapping

Exemples attendus :

male		FHIR AdministrativeGender		male→M		M		équivalence
IMP		v3-ActCode		IMP→I		I		mapping sémantique

Ajouter éventuellement la transformation de date comme exemple de **transformation syntaxique**, afin de bien distinguer syntaxe et sémantique.

Vérification

Confirmer qu'aucun mapping n'est obtenu par simple ressemblance textuelle.

Pour chaque mapping, répondre :

1. Quel est le système source ?
2. Quel est le concept ?
3. Quel est le système cible ?
4. Le sens est-il conservé ?
5. Y a-t-il perte ou ambiguïté ?

Si oui, la rendre visible dans l'interface.

Ce prompt est particulièrement bon pour montrer que tu as compris **MCLB4 : syntaxe ≠ sémantique**.

Prompt 7 — Erreurs, traçabilité, tests et audit final

Le dernier prompt doit montrer que tu ne t'arrêtes pas au « ça fonctionne ».

Nous avons maintenant le parcours principal.

Avant de considérer le prototype terminé, réalise une **revue de conformité complète par rapport au cahier des charges et au sujet de l'évaluation**.

1. Gestion minimale des erreurs

Vérifie au minimum :

- serveur indisponible ;
- HTTP 4xx ;
- HTTP 5xx ;
- Patient inexistant ;
- réponse sans `entry` ;
- `resourceType` inattendu ;
- `OperationOutcome` ;
- création Encounter refusée ;
- champ nécessaire au mapping absent ;
- mapping sémantique impossible.

Aucune erreur ne doit produire un faux message de succès.

2. Traçabilité

Ajouter une traçabilité technique minimale :

date/heure

méthode HTTP

endpoint

type de ressource

code HTTP

résultat

erreur éventuelle

Ne pas enregistrer inutilement les informations personnelles du Patient dans les logs.

Les logs peuvent être affichés dans une zone technique discrète ; ils ne doivent pas complexifier le MVP.

3. Test de généricité

Effectuer le parcours avec au moins deux Patients différents du serveur.

Vérifier :

- aucune valeur Patient codée en dur ;
- aucune dépendance à un id particulier ;
- champs absents correctement gérés ;
- HL7 évolue réellement avec les données ;
- mappings restent corrects.

4. Matrice de vérification

Produis une matrice finale :

Exigence | Implémentée ? | Test réalisé | Résultat |
Correction restante

avec obligatoirement :

- connexion FHIR réelle ;
- Patient réel ;
- création Encounter réelle ;
- vue métier ;
- vue FHIR ;
- vue HL7 ;
- vue mapping ;
- erreurs ;
- traçabilité ;
- test multi-patient.

5. Revue critique

Recherche activement :

- donnée codée en dur ;
- valeur inventée ;
- mapping non justifié ;
- fonctionnalité inutile ;
- code dupliqué ;
- erreur masquée ;
- mauvaise relation Patient/Encounter ;
- confusion entre donnée métier et donnée technique.

Pour chaque problème :

problème → cause → correction → test de non-régression.

6. Conclusion

Ne considère le prototype **GO** que si les exigences obligatoires sont démontrables avec les données réellement obtenues depuis HAPI.

Termine par :

GO / NO-GO

et indique les éventuelles limites restantes.

Prompt:

selon ces prompts, génère moi la réalisation

Sur github: fhir-admission-prototype